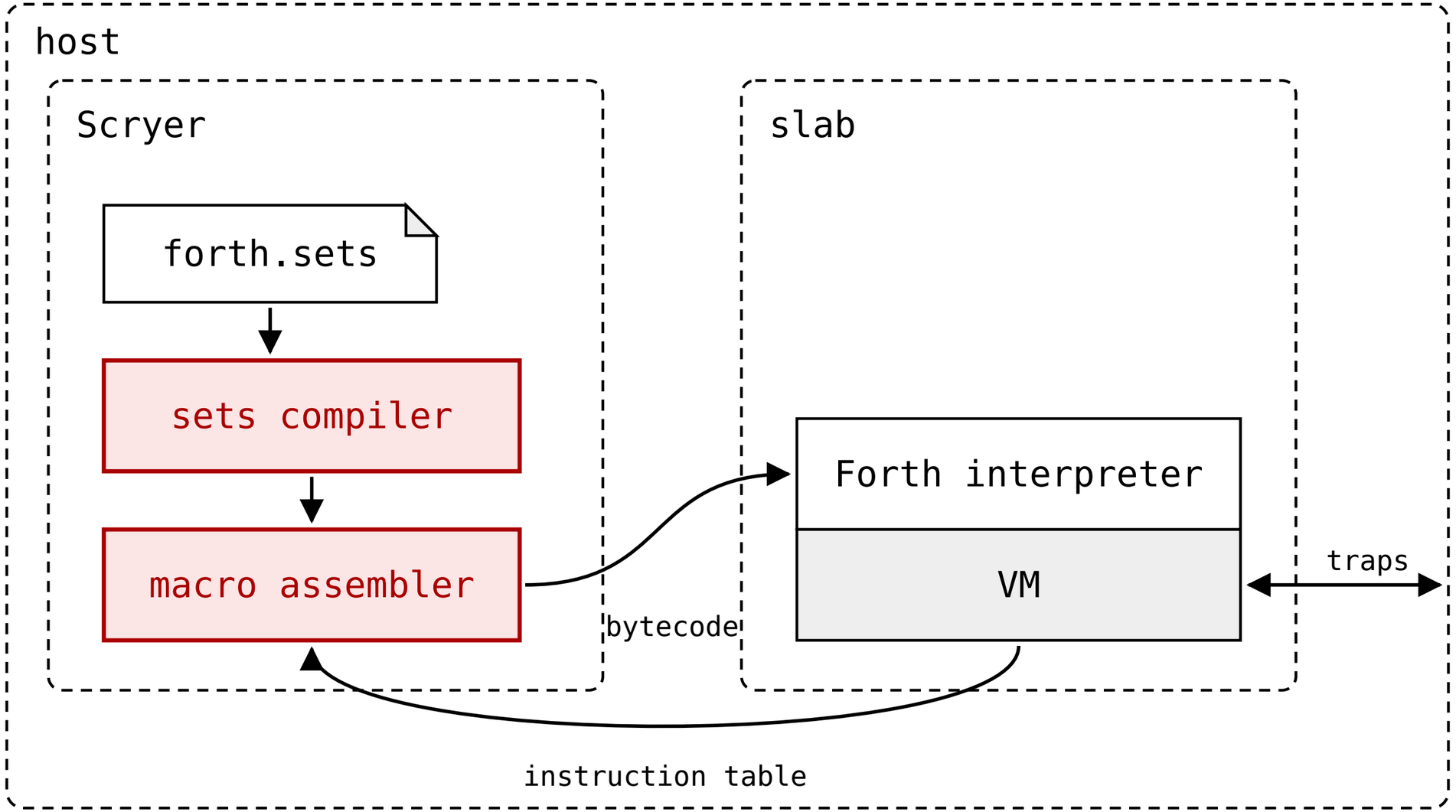
slab + sets

Gergely Harasztos

https://github.com/no382001/slab

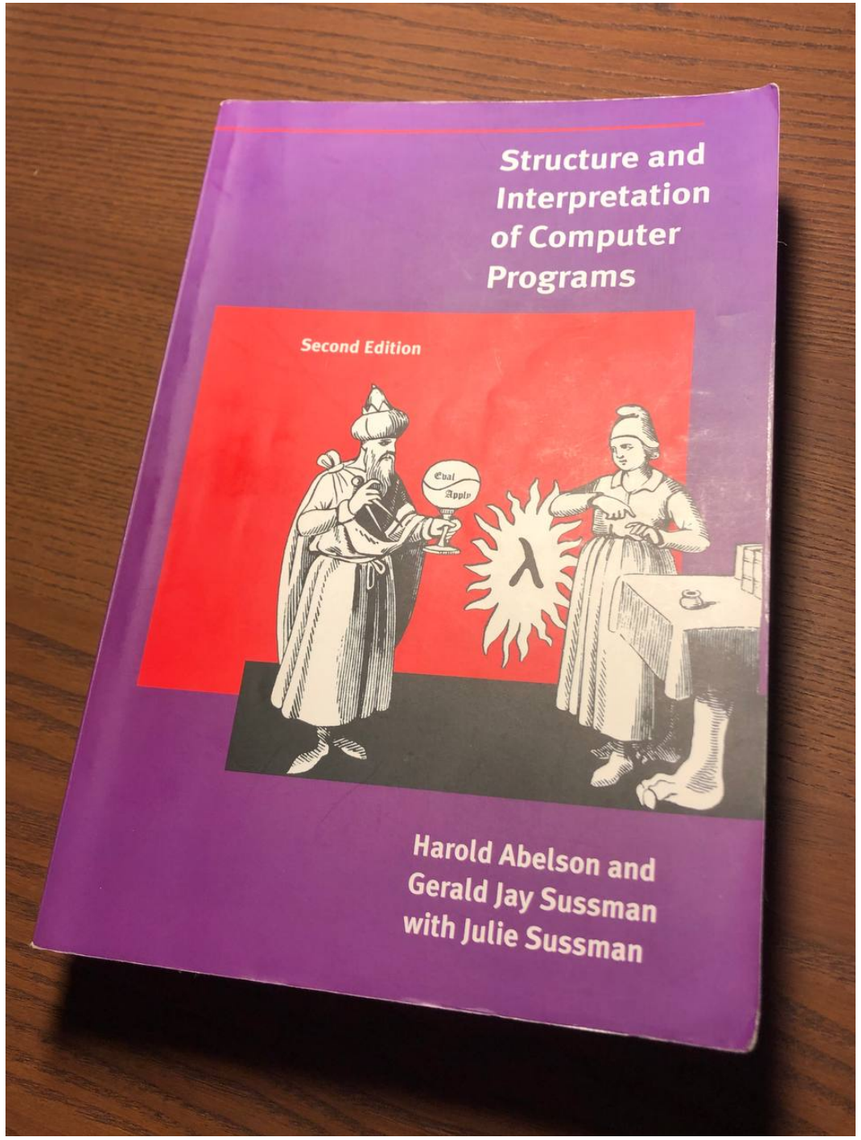
Scryer Prolog Meetup 2026, Vienna

architecture overview



how I got into Prolog

- "C++ templates are Turing complete"
- the metacircular evaluator
- templates are very rigid



how I got into Prolog

- Markus Triska's The Power of Prolog



templates vs Prolog

prolog

```prolog
len([], 0).
len([_|Xs], N) :- len(Xs, N0), N is N0 + 1.
```

c++

```cpp
template<class...> struct list {};

template<class L> struct len;

template<> struct len<list<>> {
    static constexpr int value = 0;
};

template<class X, class... Xs> struct len<list<X, Xs...>> {
    static constexpr int value = 1 + len<list<Xs...>>::value;
};
```

previous projects using the same pattern

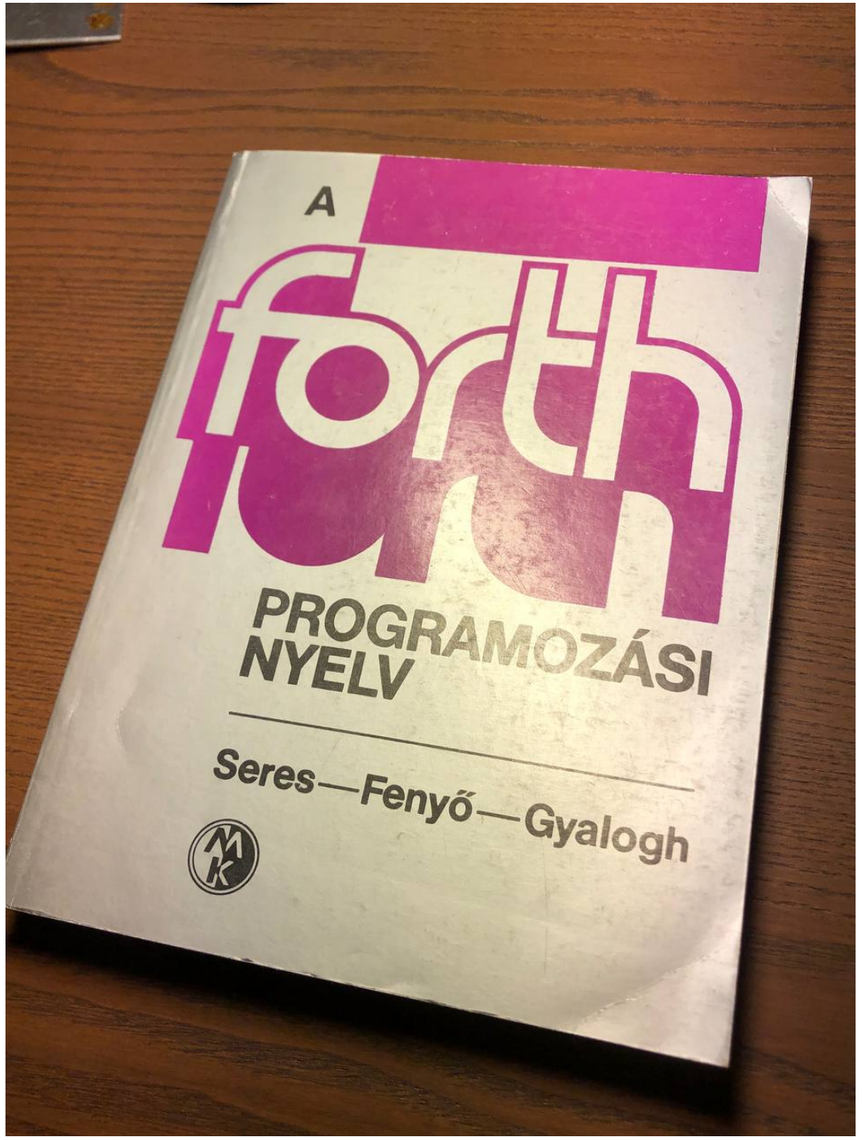
- a small C-like language targeting CHIP8
- ported to my PDP-11 emulator: bootstrapper, paper tape loader

what is slab good for?

- embeddable
- simple
- snapshottable
- isolated
- small
- extensible through traps

a bit about Forth

- simple to implement
- small
- flexible
- word-sized abstractions
- reflective

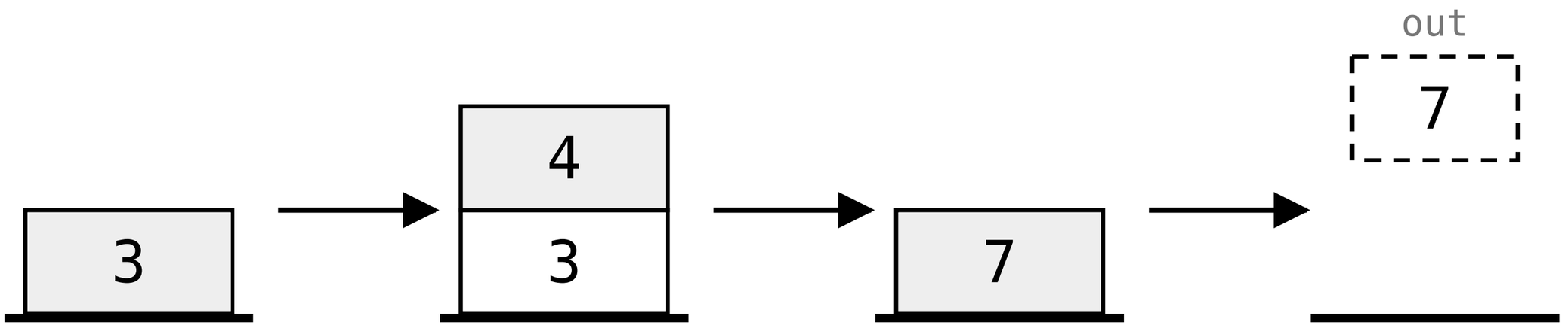


Forth

- stack machine
- words
- the dictionary

a stack machine

3 4 + .



the dictionary

LATEST →

name	code
sq	dup 1 +
sq	dup *
:	primitive
…	…
dup	primitive
bye	primitive

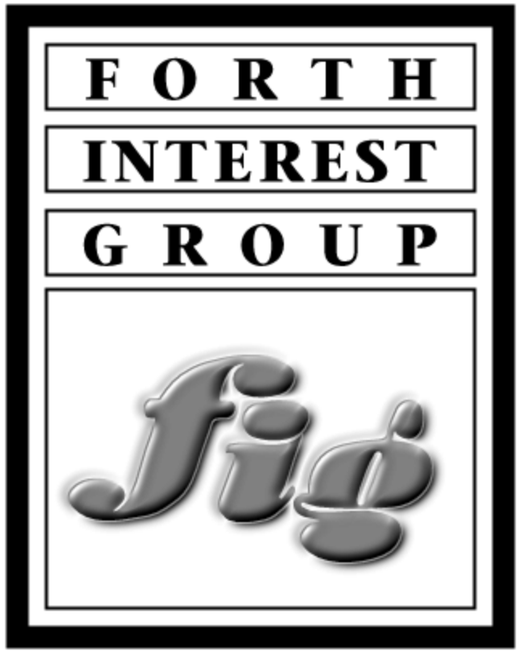
apparent problems

```forth
( n -- f )
: fib
  dup 1 > if
    dup 1 - recurse
    swap 2 - recurse +
  then ;
```

- stack discipline
- structuring a project

from interpreter to VM

- 1st: forgot the return stack
- other implementations: everything is a vm
- metacompilers
- a reduced instruction set, just enough to bootstrap a REPL

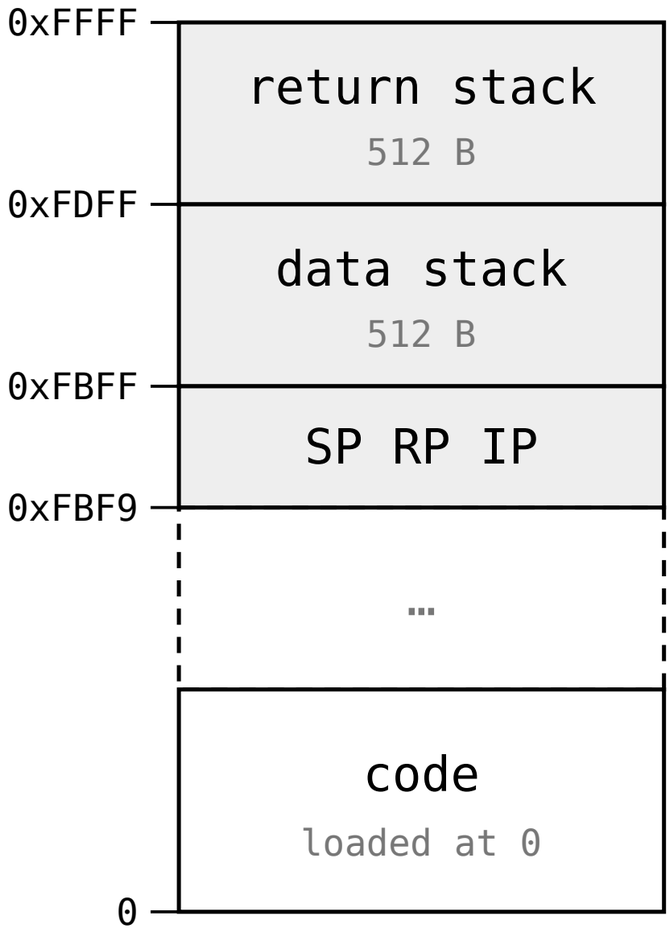


what I wanted

- simple as possible
- the system has access to its own internals
- safely word addressable for reflection

slab

the VM


0xFFFF
return stack
512 B
0xFDFF
data stack
512 B
0xFBFF
SP RP IP
0xFBF9
…
code
loaded at 0
0

building slab

writing bytes by hand

```c
uint8_t program[] = {
    OP_LIT, 3,   // push 3
    OP_LIT, 4,   // push 4
    OP_ADD,      // pop two, push the sum
    OP_DOT,      // print
    OP_HALT,
};
```

- parser generators? hand written?

building slab

first steps

```prolog
% op(Name, Opcode, StackIn, StackOut, RStackIn, RStackOut)
op('lit', 1, 0, 1, 0, 0).
op('dup', 7, 1, 2, 0, 0).
op('swap', 8, 2, 2, 0, 0).
```

- the VM prints its own instruction table as Prolog facts
- the frontend depends on the VM

building slab

the assembler

```prolog
assemble([]) --> [].
assemble([N|T]) --> { number(N), op(lit, Op, _, _, _, _) }, !,
    [Op, N], assemble(T).
assemble([A|T]) --> { op(A, Op, _, _, _, _) }, !,
    [Op], assemble(T).
assemble([A|_]) --> { domain_error(instruction, A, assemble//1) }.
```

```prolog
?- phrase(assemble([3, 4, add, dot, halt]), Bytes).
   Bytes = [1,3,1,4,2,6,0].
```

building slab

jumping to hand-counted addresses

forth

```forth
\ ex 1.
key emit branch 0

\ ex 2.
label(main)
    key emit
    branch(main)
```

building slab

labels 1

```prolog
emit_binary(Tokens, Bytes) :-
    phrase(labels(Tokens, 0), Labels),
    phrase(encode(Tokens, Labels), Bytes).

% pair labels with their offset
labels([], _) --> [].
labels([label(N)|Ts], P) --> !, [N-P], labels(Ts, P).
labels([T|Ts], P0) --> { token_size(T, S) }, !,
    { P is P0 + S }, labels(Ts, P).
labels([T|_], _) --> { domain_error(token, T, labels//2) }.
```

building slab

labels 2

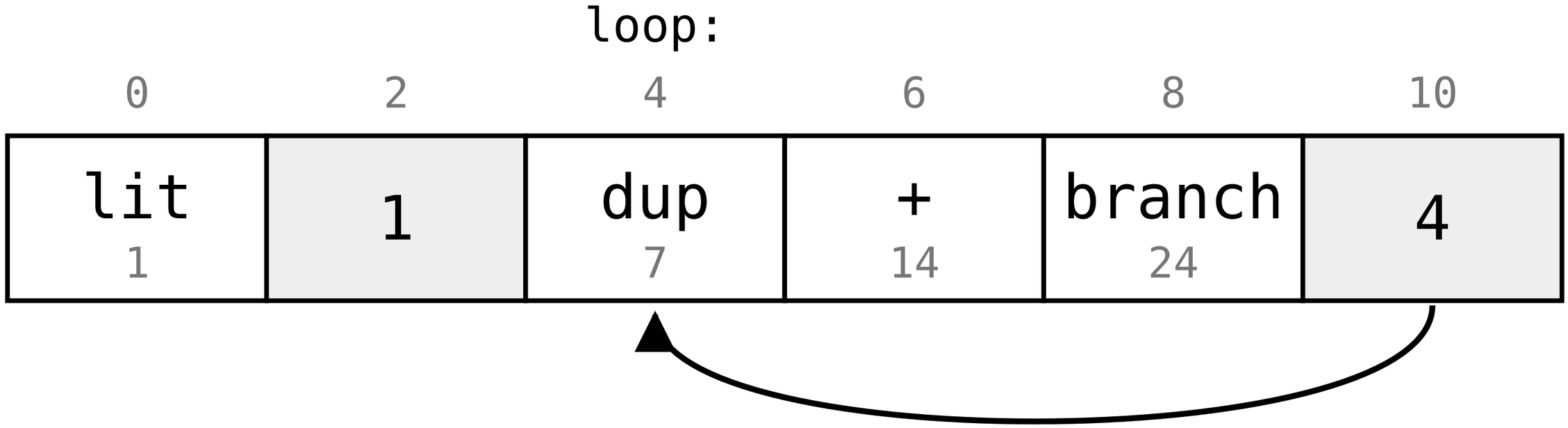
prolog

```
% replace each jump target with its offset
encode([branch(L)|Ts], Ls) -->
    { opcode(branch, Op), memberchk(L-Addr, Ls) }, !,
    [Op, Addr], encode(Ts, Ls).
encode([branch(L)|_], _) -->
    { throw(error(existence_error(label, L), encode//2)) }.
```

building slab

prolog

```
[lit, 1, label(loop), dup, +, branch(loop)]
```



building slab

macros

```prolog
expand([], _) --> [].
expand([def(N, V)|T], Ds) --> !, expand(T, [N-V|Ds]).
expand([subst(N)|T], Ds) --> { memberchk(N-V, Ds) }, !,
    [V], expand(T, Ds).
expand([subst(N)|_], _) -->
    { throw(error(existence_error(macro, N), expand//2)) }.
expand([X|T], Ds) --> [X], expand(T, Ds).
```

```forth
def(ptr, 255)
subst(ptr) c@     \ becomes: 255 c@
```

building slab

still unreadable

forth

```
\ from programs/stdlib.ma
label(strcmp)
label(strcmp_loop)
    over c@                 \ ( a1 a2 c1 )
    over c@                 \ ( a1 a2 c1 c2 )
    over over =             \ ( a1 a2 c1 c2 equal? )
    zbranch(strcmp_ne)
    drop                    \ ( a1 a2 c1 )
    zbranch(strcmp_eq)
    swap lit 1 + swap       \ ( a1+1 a2 )
    lit 1 +                 \ ( a1+1 a2+1 )
    branch(strcmp_loop)
\ ... strcmp_eq and strcmp_ne: drop the addresses, push the flag, ret
```

sets

(s)-(e)xpression (t)yped (s)ystems language

sets

```
(def streq ((a : (ptr byte)) (b : (ptr byte))) : bool [semidet]
  (local ((i 0))
    (while (and (= (byte-ref a i) (byte-ref b i))
                (!= (byte-ref a i) 0))
      (! i (+ (@ i) 1)))
    (= (byte-ref a i) (byte-ref b i))))
```

- an expression returns one value or nothing
- `(+ 1 (* 2 3))` -> `1 2 3 * +`
- effect inference for clarity and optimizations, not det-semidet-nondet in a way like in Prolog!
- needs lots of bookkeeping

sets

```prolog
form(list(Es)) --> "(", ws, elems(Es), ")".
form(num(N))   --> num_chars(Cs), { number_chars(N, Cs) }.
form(sym(S))   --> sym_chars(Cs), { atom_chars(S, Cs) }.

elems([E|Es])  --> form(E), ws, elems(Es).
elems([])      --> [].
```

```prolog
?- phrase(form(F), "(+ 1 2)").
   F = list([sym(+),num(1),num(2)]).
```

sets

```prolog
?- parse("(def f ((x : int)) : int (if (< x 1) 0 (* x 2)))", ok(F)),
   transform_program(F, ok(Defs)).
   Defs = [def(f,
             [param(x,int)],
             int,none,
                 [if(binop(<,var(x),num(1)),num(0),binop(*,var(x),num(2)))])].
```

sets

composing

```prolog
compile_expr(if(Cond, Then, Else), Code) :-
    genlabel("_else_", ElseL),
    genlabel("_endif_", EndL),
    compile_expr(Cond, CC),
    compile_expr(Then, TC),
    compile_expr(Else, EC),
    phrase(( seq(CC),  [zbranch(ElseL)],
             seq(TC),  [branch(EndL), label(ElseL)],
             seq(EC),  [label(EndL)] ), Code).
```

types

```prolog
infer(_, num(_), int).
infer(Env, var(X), T) :- member(var(X, T), Env).
infer(Env, @(P), T) :- check(Env, P, ptr(T)).
infer(Env, binop(<, A, B), bool) :-
    check(Env, A, int), check(Env, B, int).
infer(Env, if(C, Then, Else), T) :-
    check(Env, C, bool), infer(Env, Then, T), check(Env, Else, T).

check(Env, E, T) :- infer(Env, E, T), !.
check(_, E, T) :- throw(error(type_error(T, E), _)).
```

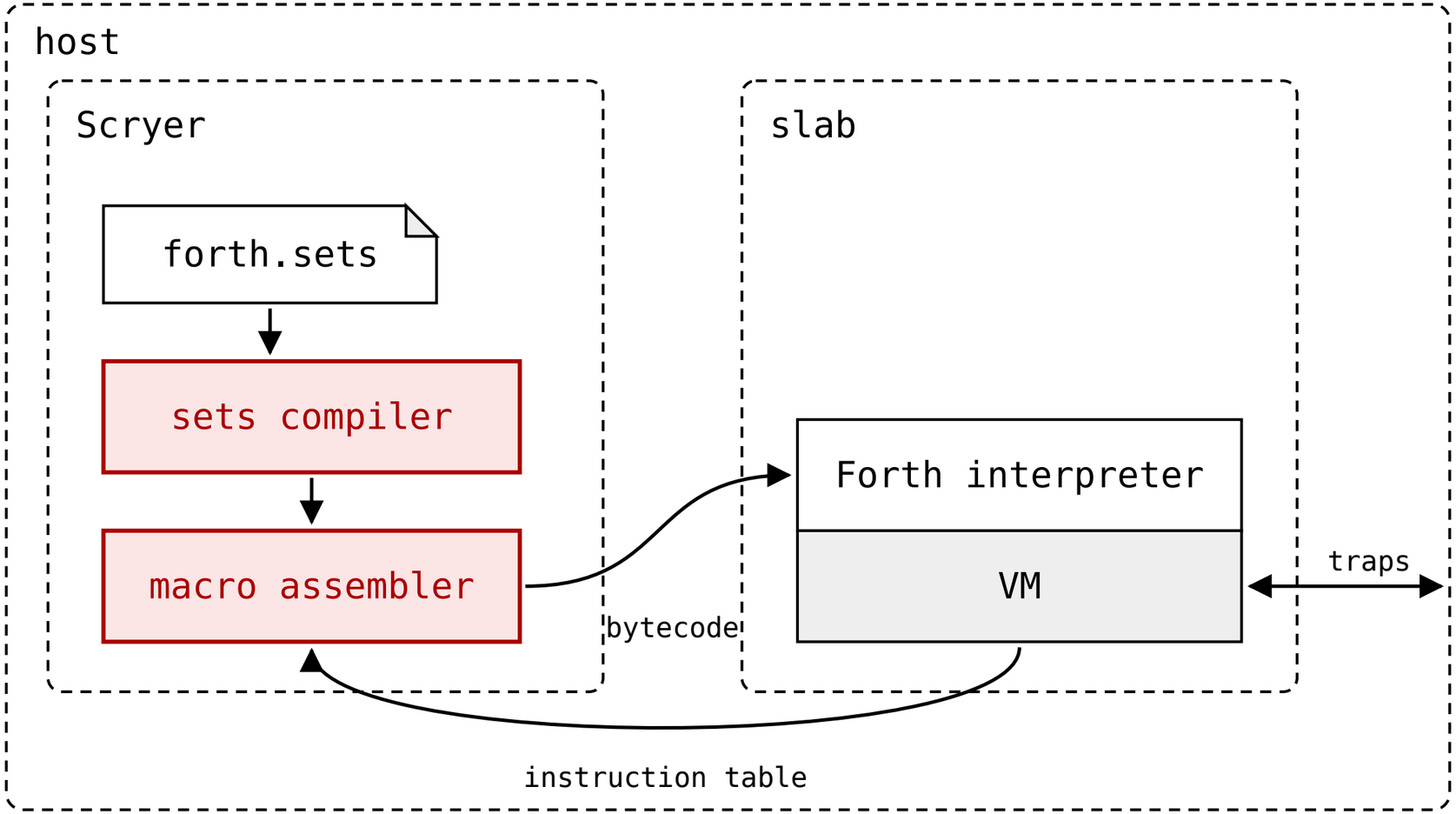
```prolog
?- infer([var(p, P), var(i, I)],
         if(binop(<, var(i), num(1)), @(var(p)), num(0)), T).
   P = ptr(int), I = int, T = int.
```

pipeline

- **locals**: `(local ...)` becomes per-function memory cells
- **typecheck**: infer and check types
- **effects**: infer det / semidet / nondet, check the annotations
- **inline**: expand calls to functions marked `[inline]`
- **constfold**
- **cse**: hoist repeated det expressions, only when it saves bytes
- **dead code elim.**: drop definitions nothing reaches anymore
- **codegen**: ast -> assembly
- **emit**: labels -> addresses, bytes out

where it ended up

the architecture



demo: Mandelbrot

thank you